

Assignment 2 - 320: Digital Therapy Assistant Requirements

Project Overview

Learning Objectives

Ethical Considerations

Deliverable 1: CLI Service For User Interaction (5 points)

Functional Requirements

Technical Requirements

CLI Menu Structure

Main Menu

Deliverables

Grading Rubric (5 points)

Deliverable 2: Spring Boot Application Implementation (5 points)

Project Structure

Core Services to Implement

AuthService

SessionService

DiaryService

Configuration Requirements

Deliverables

Grading Rubric (5 points)

Deliverable 3: REST API Implementation (5 points)

API Endpoints to Implement

Authentication Endpoints

Session Endpoints

Diary Endpoints

Progress Endpoints

Crisis Endpoints

API Requirements

Error Response Format

Deliverables

Grading Rubric (5 points)

Deliverable 5: Database Persistence (5 points)

Database Configuration

Entity Model

Relationship Mapping

Repository Layer

Data Initialization

Create a data.sql file to seed

Deliverables

Grading Rubric (5 points)

Deliverable 6: AI Service AND RAG Implementation (5 points)

LLM Provider Integration

AI Service Interface

RAG Implementation with SimpleVector

Vector Store Service

Knowledge Base Content

Create a knowledge base containing CBT reference materials

Context Retrieval

Prompt Templates

- Therapeutic Chat System Prompt
- Distortion Analysis Prompt
- Return format
- Crisis Detection Prompt
- Return JSON
- Crisis Detection Implementation
- Deliverables
- Grading Rubric (5 points)
- PART 6: C4 Architecture Diagrams (5 points)
 - Required Diagrams
 - Level 1: System Context Diagram (c4-context.puml)
 - Users
 - External Systems
 - Level 2: Container Diagram (c4-container.puml)
 - Show the high-level technical building blocks
 - Level 3: Component Diagram (c4-component.puml)
 - Detail the Application components
 - Level 4: Code Diagrams (c4-code.puml)
 - Diagram Requirements
 - Grading Rubric (5 points)
- Submission Guidelines
 - Project Structure
- Appendix
 - Appendix A: Properties
 - application.properties
 - Appendix B: Knowledge Base Sample Files
 - distortions.json

Project Overview

Digital Therapy Assistant is an AI-guided Cognitive Behavioral Therapy (CBT) platform designed to support workplace burnout recovery. This assignment challenges you to implement key components of this Digital Therapy Assistant system using modern software engineering practices

You will build a comprehensive system that includes

- Command-line interface for user interaction
- Spring Boot backend application
- RESTful API endpoints
- Database persistence with JPA and H2
- AI-powered features using RAG with SimpleVector storage
- Architectural documentation using C4 diagrams

Learning Objectives

Upon completion of this assignment, you will be able to

1. Design and implement a CLI application following SOLID principles
2. Build a Spring Boot application with proper layered architecture
3. Implement RESTful APIs following OpenAPI specifications
4. Apply JPA/Hibernate for database persistence with proper entity relationships
5. Integrate AI services (Claude.ai, OpenAI, or Gemini) using RAG patterns
6. Document software architecture using C4 model diagrams including code-level views

Ethical Considerations

This project involves mental health technology. Keep these principles in mind

- Privacy & Confidentiality: All designs must consider HIPAA compliance concepts
- Crisis Handling: Include crisis detection and pathways to emergency resources
- Clinical Validity: Reference evidence-based CBT techniques
- Inclusive Design: Consider diverse mental health needs and accessibility

Deliverable 1: CLI Service For User Interaction (5 points)

Implement a command-line interface that allows users to interact with the Digital Therapy Assistant system. The CLI should provide a menu-driven experience supporting core therapeutic workflows

Functional Requirements

1. User Authentication: Register, login, and logout commands with secure password handling
2. Session Management: Start, continue, and end CBT sessions
3. Thought Diary: Create, view, and manage thought record entries
4. Progress Tracking: Display weekly/monthly progress summaries
5. Crisis Support: Quick access to crisis resources and safety plan

Technical Requirements

- Use the Command Pattern for menu actions
- Implement input validation with meaningful error messages
- Use dependency injection for service layer access
- Implement proper exception handling with user-friendly messages

CLI Menu Structure

Main Menu

1. Authentication
 - a. Register

- b. Login
- c. Logout
- 2. CBT Sessions
 - a. View Session Library
 - b. Start New Session
 - c. View Session History
- 3. Thought Diary
 - a. New Entry
 - b. View Entries
 - c. View Insights
- 4. Progress Dashboard
 - a. Weekly Summary
 - b. Monthly Trends
 - c. Achievements
- 5. Crisis Support
 - a. Coping Strategies
 - b. Emergency Resources
 - c. Safety Plan
- 6. Settings
- 7. Exit

Deliverables

- CLI application source code in src/main/java/cli/ package
- Command interface and concrete command implementations
- Unit tests for command execution (minimum 80% coverage)
- README documenting CLI usage

Grading Rubric (5 points)

Criterion	5 Points
Command Pattern Implementation	1.5
Menu Navigation & UX	1.5
Input Validation & Error Handling	1.0
Unit Test Coverage	1.0

Deliverable 2: Spring Boot Application Implementation (5 points)

Build the core Spring Boot application that serves as the backbone of the Digital Therapy Assistant platform. This component implements the business logic layer and coordinates between the CLI and data layers

Project Structure

```
1 src/main/java/com/digitaltherapy/  
2   ├── DigitalTherapyAssistantApplication.java  
3   ├── config/  
4       ├── SecurityConfig.java  
5       ├── JpaConfig.java  
6       └── AiServiceConfig.java  
7   ├── cli/  
8       ├── Command.java  
9       ├── MenuHandler.java  
10      ├── DigitalTherapyCLIClient.java  
11      └── commands/  
12  ├── controller/  
13      ├── AuthController.java  
14      ├── SessionController.java  
15      ├── DiaryController.java  
16      ├── ProgressController.java  
17      └── CrisisController.java  
18  ├── service/  
19      ├── AuthService.java  
20      ├── SessionService.java  
21      ├── DiaryService.java  
22      ├── ProgressService.java  
23      ├── CrisisService.java  
24      └── AiService.java  
25  ├── repository/  
26  ├── entity/  
27  ├── dto/  
28  └── exception/
```

Core Services to Implement

AuthService

```
1 public interface AuthService {  
2     AuthResponse register(RegisterRequest request);  
3     AuthResponse login(LoginRequest request);  
4     AuthResponse refreshToken(String refreshToken);  
5     void logout(String accessToken);  
6 }
```

SessionService

```
1 public interface SessionService {  
2     List<SessionModule> getSessionLibrary(UUID userId);  
3     SessionDetail getSessionDetails(UUID sessionId);  
4     ActiveSession startSession(UUID userId, UUID sessionId);  
5     SessionSummary endSession(UUID sessionId, String reason);  
6     List<SessionHistoryEntry> getSessionHistory(UUID userId);  
7 }
```

DiaryService

```
1 public interface DiaryService {
```

```

2     DiaryEntryResponse createEntry(UUID userId, DiaryEntryCreate
    request);
3     Page<DiaryEntrySummary> getEntries(UUID userId, Pageable pageable);
4     DiaryEntryDetail getEntryDetail(UUID entryId);
5     List<DistortionSuggestion> suggestDistortions(String thought);
6     DiaryInsights getInsights(UUID userId);
7 }

```

Configuration Requirements

- JWT-based authentication configuration with Spring Security
- Database connection with H2 (file-based for persistence)
- Proper logging configuration with SLF4J
- Exception handling with @ControllerAdvice

Deliverables

- Complete Spring Boot application with all layers
- Single application.properties configuration file
- Service layer unit tests (minimum 85% coverage)
- Integration tests for critical workflows

Grading Rubric (5 points)

Criterion	5 Points
Layered Architecture	1.5
Dependency Injection	1.0
Service Implementation	1.5
Configuration & Security	0.5
Test Coverage	0.5

Deliverable 3: REST API Implementation (5 points)

Implement the RESTful API endpoints based on the provided OpenAPI specification. Your API should follow REST best practices and return responses that match the specification exactly

API Endpoints to Implement

Authentication Endpoints

Method	Endpoint	Description
POST	/auth/register	Register new user

POST	/auth/login	Authenticate user
POST	/auth/refresh	Refresh access token
POST	/auth/logout	Invalidate session

Session Endpoints

Method	Endpoint	Description
GET	/sessions	Get session library
GET	/sessions/{sessionId}	Get session details
POST	/sessions/{sessionId}/start	Start CBT session
POST	/sessions/{sessionId}/chat	Send chat message
POST	/sessions/{sessionId}/end	End session

Diary Endpoints

Method	Endpoint	Description
GET	/diary/entries	List diary entries
POST	/diary/entries	Create diary entry
GET	diary/entries/{entryId}	Get entry detail
DELETE	/diary/entries/{entryId}	Delete entry
POST	diary/distortions/suggest	AI suggest distortions
GET	/diary/insights	Get pattern insights

Progress Endpoints

Method	Endpoint	Description
GET	/progress/weekly	Get weekly progress

GET	/progress/monthly	Get monthly trends
GET	/progress/burnout	Get burnout recovery
GET	/progress/achievements	Get achievements

Crisis Endpoints

Method	Endpoint	Description
GET	/crisis	Get crisis hub
GET	/crisis/coping-strategies	Get coping strategies
POST	/crisis/detect	Detect crisis indicators
GET	/crisis/safety-plan	Get safety plan
PUT	/crisis/safety-plan	Update safety plan

API Requirements

- Use proper HTTP status codes (200, 201, 400, 401, 403, 404, 409, 422)
- Implement request validation with Bean Validation (JSR-380)
- Support pagination for list endpoints
- Include proper error response format with error codes and messages
- Document all endpoints with OpenAPI annotations

Error Response Format

```

1 {
2   "error": {
3     "code": "VALIDATION_ERROR",
4     "message": "Invalid input provided",
5     "details": [
6       {
7         "field": "email",
8         "message": "must be a valid email address"
9       }
10    ],
11    "timestamp": "2026-01-15T10:30:00Z"
12  }
13 }
```

Deliverables

- All controller classes with endpoint implementations

- Request/Response DTOs with validation annotations
- OpenAPI documentation (generated via springdoc-openapi)
- API integration tests

Grading Rubric (5 points)

Criterion	5 Points
Endpoint Implementation	2.0
OpenAPI Compliance	1.0
Validation & Error Handling	1.0
API Documentation	1.0

Deliverable 5: Database Persistence (5 points)

Design and implement the database layer using JPA/Hibernate with H2 database. Create proper entity relationships, implement repository patterns, and ensure data integrity

Database Configuration

Use H2 database with file-based persistence. See application.properties in Appendix A.

Entity Model

```

1 User Entity:
2 id: UUID (PK)
3 email: String (unique, not null)
4 passwordHash: String (not null)
5 name: String
6 onboardingComplete: Boolean
7 onboardingPath: Enum (SELF, THERAPIST_REFERRED)
8 severityLevel: Enum (MILD, MODERATE, SIGNIFICANT)
9 streakDays: Integer
10 createdAt: LocalDateTime
11 updatedAt: LocalDateTime

```

```

1 CbtSession Entity:
2 id: UUID (PK)
3 module: SessionModule (ManyToOne)
4 title: String
5 description: String (TEXT)
6 durationMinutes: Integer
7 objectives: List<String> (ElementCollection)
8 modalities: List<Enum> (ElementCollection)
9 orderIndex: Integer

```

```

1 UserSession Entity:
2 id: UUID (PK)
3 user: User (ManyToOne)
4 cbtSession: CbtSession (ManyToOne)
5 status: Enum (IN_PROGRESS, COMPLETED, EARLY_EXIT)

```

```
6 startedAt: LocalDateTime
7 endedAt: LocalDateTime
8 moodBefore: Integer (1-10)
9 moodAfter: Integer (1-10)
```

```
1 DiaryEntry Entity:
2 id: UUID (PK)
3 user: User (ManyToOne)
4 situation: String (TEXT)
5 automaticThought: String (TEXT)
6 emotions: List<EmotionRating> (ElementCollection)
7 distortions: List<CognitiveDistortion> (ManyToMany)
8 alternativeThought: String (TEXT)
9 moodBefore: Integer
10 moodAfter: Integer
11 beliefRatingBefore: Integer
12 beliefRatingAfter: Integer
13 createdAt: LocalDateTime
14 deleted: Boolean (soft delete)
```

```
1 ChatMessage Entity:
2 id: UUID (PK)
3 userSession: UserSession (ManyToOne)
4 role: Enum (USER, ASSISTANT)
5 content: String (TEXT)
6 modality: Enum (TEXT, VOICE, VIDEO)
7 timestamp: LocalDateTime
```

```
1 CognitiveDistortion Entity:
2 id: String (PK)
3 name: String
4 description: String (TEXT)
5 examples: List<String> (ElementCollection)
```

```
1 TrustedContact Entity:
2 id: UUID (PK)
3 user: User (ManyToOne)
4 name: String
5 phone: String
6 relationship: String
```

Relationship Mapping

1. User to UserSession: One-to-Many (user owns sessions)
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
2. User to DiaryEntry: One-to-Many (user owns diary entries)
@OneToMany(mappedBy = "user", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
3. SessionModule to CbtSession: One-to-Many (module contains sessions)
@OneToMany(mappedBy = "module", cascade = CascadeType.ALL)
4. UserSession to ChatMessage: One-to-Many (session contains messages)
@OneToMany(mappedBy = "userSession", cascade = CascadeType.ALL)
5. DiaryEntry to CognitiveDistortion: Many-to-Many (entries tag distortions)
@ManyToMany
@JoinTable(name = "diary_entry_distortions")

6. User to TrustedContact: One-to-Many (user has emergency contacts)

@OneToMany(mappedBy = "user", cascade = CascadeType.ALL)

Repository Layer

Implement Spring Data JPA repositories with custom queries

```
1 public interface UserSessionRepository extends
  JpaRepository<UserSession, UUID> {
2
3     List<UserSession> findByUserIdOrderByStartedAtDesc(UUID userId);
4
5     @Query("SELECT us FROM UserSession us WHERE us.user.id = :userId " +
6           "AND us.startedAt >= :startDate AND us.startedAt < :endDate")
7     List<UserSession> findByUserIdAndDateRange(
8         @Param("userId") UUID userId,
9         @Param("startDate") LocalDateTime startDate,
10        @Param("endDate") LocalDateTime endDate);
11
12    @Query("SELECT COUNT(us) FROM UserSession us " +
13          "WHERE us.user.id = :userId AND us.status = 'COMPLETED'")
14    Long countCompletedByUserId(@Param("userId") UUID userId);
15
16    Optional<UserSession>
17    findFirstByUserIdAndStatusOrderByStartedAtDesc(
18        UUID userId, SessionStatus status);
19 }
```

```
1 public interface DiaryEntryRepository extends
  JpaRepository<DiaryEntry, UUID> {
2
3     Page<DiaryEntry> findByUserIdAndDeletedFalseOrderByCreatedAtDesc(
4         UUID userId, Pageable pageable);
5
6     @Query("SELECT d.distortions, COUNT(d) FROM DiaryEntry d " +
7           "WHERE d.user.id = :userId AND d.deleted = false " +
8           "GROUP BY d.distortions ORDER BY COUNT(d) DESC")
9     List<Object[]> findTopDistortionsByUserId(@Param("userId") UUID
10        userId);
11
12    @Query("SELECT AVG(d.moodAfter - d.moodBefore) FROM DiaryEntry d " +
13          "WHERE d.user.id = :userId AND d.deleted = false")
14    Double calculateAverageMoodImprovement(@Param("userId") UUID
15        userId);
16 }
```

Data Initialization

Create a data.sql file to seed

- Cognitive distortion definitions (all-or-nothing, catastrophizing, etc.)
- Sample CBT session modules and sessions
- Coping strategies

Deliverables

- All JPA entity classes with proper annotations

- Repository interfaces with custom queries
- schema.sql and data.sql scripts
- Entity relationship diagram (ERD)

Grading Rubric (5 points)

Criterion	Points
Entity Design & Relationships	2.0
Repository Implementation	1.5
Schema Scripts	0.5
Data Integrity & Auditing	1.0

Deliverable 6: AI Service AND RAG Implementation (5 points)

Implement the AI-powered features using Retrieval-Augmented Generation (RAG) patterns. The AI service should provide context-aware therapeutic responses based on CBT principles and user history

LLM Provider Integration

Choose ONE of the following LLM providers to integrate:

Option A:

 **Claude**
 Claude is Anthropic's AI, built for problem solvers. Tackle complex challenges, analyze data, write code, and think through your hardest work.
 Claude


 Claude
BY ANTHROPIC

(Anthropic)

- Use the Anthropic Java SDK or REST API
- Model: claude-3-sonnet or claude-3-haiku
- Endpoint: <https://api.anthropic.com/v1/messages>

Option B: OpenAI

- Use the OpenAI Java SDK or REST API
- Model: gpt-4 or gpt-3.5-turbo
- Endpoint: <https://api.openai.com/v1/chat/completions>

Option C: Gemini (Google)

- Use the Google AI Java SDK or REST API
- Model: gemini-pro
- Endpoint: <https://generativelanguage.googleapis.com/v1/models>

AI Service Interface

```
1 public interface AiService {
2
3     // Generate therapeutic chat response
4     ChatResponse generateResponse(UUID sessionId, String userMessage);
5
6     // Suggest cognitive distortions from thought
7     List<DistortionSuggestion> analyzeThought(String automaticThought);
8
9     // Generate reframing prompts
10    List<String> generateReframingPrompts(String thought, List<String>
distortionIds);
11
12    // Detect crisis indicators in text
13    CrisisDetectionResult detectCrisis(String text);
14
15    // Generate insights from diary entries
16    DiaryInsights generateInsights(UUID userId);
17
18    // Summarize session for completion
19    SessionSummary summarizeSession(UUID sessionId);
20 }
```

RAG Implementation with SimpleVector

Use SimpleVector for vector storage and similarity search

Vector Store Service

```
1 public class SimpleVectorStore {
2     private Map<String, float[]> vectors = new ConcurrentHashMap<>();
3     private Map<String, Document> documents = new ConcurrentHashMap<>
4     ();
5     public void addDocument(String id, Document doc, float[] embedding)
6     {
7         vectors.put(id, embedding);
8         documents.put(id, doc);
9     }
10    public List<Document> similaritySearch(float[] queryEmbedding, int
topK) {
11        return vectors.entrySet().stream()
12            .map(e -> new ScoredDocument(
13                documents.get(e.getKey()),
14                cosineSimilarity(queryEmbedding, e.getValue())
15            ))
16            .sorted(Comparator.comparingDouble(ScoredDocument::score).reversed())
17            .limit(topK)
18            .map(ScoredDocument::document)
19            .collect(Collectors.toList());
20    }
21
22    private double cosineSimilarity(float[] a, float[] b) {
```

```

23     double dotProduct = 0.0, normA = 0.0, normB = 0.0;
24     for (int i = 0; i < a.length; i++) {
25         dotProduct += a[i] * b[i];
26         normA += a[i] * a[i];
27         normB += b[i] * b[i];
28     }
29     return dotProduct / (Math.sqrt(normA) * Math.sqrt(normB));
30 }
31 }

```

Knowledge Base Content

Create a knowledge base containing CBT reference materials

1. Cognitive Distortion Definitions

- All-or-nothing thinking
- Overgeneralization
- Mental filtering
- Catastrophizing
- Mind reading
- Fortune telling
- Emotional reasoning
- Should statements
- Labeling
- Personalization

2. CBT Techniques

- Thought challenging/cognitive restructuring
- Behavioral activation
- Exposure techniques
- Relaxation and mindfulness
- Problem-solving therapy

3. Burnout-Specific Content

- Maslach Burnout Inventory dimensions
- Recovery strategies
- Work-life balance techniques
- Boundary setting

4. Crisis Intervention Protocols

- Warning signs recognition
- De-escalation techniques
- Safety planning steps

- Emergency resources

Context Retrieval

```
1 public class RagContextBuilder {
2
3     private final SimpleVectorStore vectorStore;
4     private final EmbeddingService embeddingService;
5     private final UserSessionRepository sessionRepository;
6     private final DiaryEntryRepository diaryRepository;
7
8     public String buildContext(UUID userId, UUID sessionId, String
query) {
9         StringBuilder context = new StringBuilder();
10
11         // 1. Retrieve relevant CBT knowledge
12         float[] queryEmbedding = embeddingService.embed(query);
13         List<Document> cbtDocs =
vectorStore.similaritySearch(queryEmbedding, 3);
14         context.append("## Relevant CBT Knowledge\\n");
15         cbtDocs.forEach(doc ->
context.append(doc.getContent()).append("\\n\\n"));
16
17         // 2. Get user's recent session history
18         List<UserSession> recentSessions = sessionRepository
19             .findByUserIdOrderByStartedAtDesc(userId)
20             .stream().limit(5).collect(Collectors.toList());
21         context.append("## Recent Session History\\n");
22         recentSessions.forEach(s ->
context.append(formatSession(s)).append("\\n"));
23
24         // 3. Get user's diary patterns
25         List<DiaryEntry> recentEntries = diaryRepository
26             .findByUserIdAndDeletedFalseOrderByCreatedAtDesc(userId,
PageRequest.of(0, 5))
27             .getContent();
28         context.append("## Recent Diary Patterns\\n");
29         recentEntries.forEach(e ->
context.append(formatEntry(e)).append("\\n"));
30
31         // 4. Get current session transcript
32         if (sessionId != null) {
33             List<ChatMessage> transcript =
getSessionTranscript(sessionId);
34             context.append("## Current Session Transcript\\n");
35             transcript.forEach(m ->
context.append(formatMessage(m)).append("\\n"));
36         }
37
38         return context.toString();
39     }
```

Prompt Templates

Therapeutic Chat System Prompt

```
1 You are a compassionate AI therapy assistant specializing in Cognitive
2 Behavioral Therapy (CBT) for workplace burnout recovery. Your role is
to:
3
4 Maintain an empathetic, non-judgmental, and supportive tone
5 Use Socratic questioning to help users explore their thoughts
6 Guide users through CBT techniques like thought challenging
7 Recognize and gently address cognitive distortions
8 Celebrate progress and provide encouragement
```

```

9
10 IMMEDIATELY recognize crisis indicators and respond appropriately
11
12 Context about the user and relevant CBT knowledge:
13 {context}
14
15 Remember:
16 Never provide medical diagnoses
17 Encourage professional help when appropriate
18 Maintain appropriate boundaries
19 Prioritize user safety above all else

```

Distortion Analysis Prompt

```

1 Analyze the following automatic thought for cognitive distortions.
2 Return a JSON array of identified distortions with confidence scores.
3
4 Thought: "{thought}"
5
6 Available distortion types: {distortion_list}

```

Return format

```

1 [
2   {
3     "distortionId": "all-or-nothing",
4     "confidence": 0.85,
5     "reasoning": "The thought uses absolute terms like 'always' and
6     'never'"
7   }
8 ]

```

Crisis Detection Prompt

```

1 Analyze the following text for crisis indicators. Assess risk level
2 and
3 recommend appropriate action.
4
5 Text: "{text}"
6
7 Evaluate for:
8 Suicidal ideation or self-harm mentions
9 Expressions of hopelessness
10 Statements about being a burden
11 Plans or intentions to harm self/others
12 Severe emotional distress

```

Return JSON

```

1 {
2   "riskLevel": "none|low|medium|high|critical",
3   "keywordsDetected": [],
4   "recommendedAction":
5   "none|show_resources|show_crisis_hub|immediate_intervention",
6   "reasoning": "..."
7 }

```

Crisis Detection Implementation

```

1 public class CrisisDetector {
2
3   private static final Set<String> CRISIS_KEYWORDS = Set.of(
4     "suicide", "kill myself", "end it all", "no reason to live",
5     "better off dead", "can't go on", "want to die", "hurt myself"
6   );
7 }

```

```

6   );
7
8   private final AiClient aiClient;
9
10  public CrisisDetectionResult analyze(String text) {
11      // Layer 1: Keyword-based detection (fast)
12      List<String> detectedKeywords = CRISIS_KEYWORDS.stream()
13          .filter(k -> text.toLowerCase().contains(k))
14          .collect(Collectors.toList());
15
16      // Layer 2: AI-based semantic analysis
17      RiskLevel aiRisk = performAiAnalysis(text);
18
19      // Combine signals - err on the side of caution
20      RiskLevel finalRisk = combineRiskAssessments(detectedKeywords,
21      aiRisk);
22
23      return CrisisDetectionResult.builder()
24          .riskLevel(finalRisk)
25          .keywordsDetected(detectedKeywords)
26          .recommendedAction(determineAction(finalRisk))
27          .build();
28  }
29
30  private RecommendedAction determineAction(RiskLevel risk) {
31      return switch (risk) {
32          case CRITICAL -> RecommendedAction.IMMEDIATE_INTERVENTION;
33          case HIGH -> RecommendedAction.SHOW_CRISIS_HUB;
34          case MEDIUM -> RecommendedAction.SHOW_RESOURCES;
35          default -> RecommendedAction.NONE;
36      };
37  }

```

Deliverables

- AiService implementation with chosen LLM provider
- SimpleVectorStore implementation
- RagContextBuilder for context assembly
- Knowledge base documents (JSON or text files)
- Prompt templates
- Crisis detection module
- Unit tests for AI service (mock LLM responses)

Grading Rubric (5 points)

Criterion	Points
LLM Integration	1.5
RAG with SimpleVector	1.5
Prompt Engineering	1.0
Crisis Detection	1.0

PART 6: C4 Architecture Diagrams (5 points)

Document the system architecture using the C4 model. Create diagrams at multiple levels of abstraction including code-level views with class and sequence diagrams.

Required Diagrams

Level 1: System Context Diagram (c4-context.puml)

Show Digital Therapy Assistant in context with its users and external systems:

Users

- Patients (primary users seeking burnout recovery)
- Therapists (monitoring patient progress)
- System Administrators (managing the platform)

External Systems

- LLM Provider (



- OpenAI / Gemini)
- Email Service (notifications)
- Healthcare EHR Systems (FHIR export)

Level 2: Container Diagram (c4-container.puml)

Show the high-level technical building blocks

- CLI Application (Java) - Command-line interface
- Spring Boot Application - Backend services
- H2 Database - Embedded relational database
- SimpleVector Store - Vector storage for RAG
- Knowledge Base - CBT reference documents

Level 3: Component Diagram (c4-component.puml)

Detail the Application components

Controller Layer

- AuthController
- SessionController

- DiaryController
- ProgressController
- CrisisController

Service Layer

- AuthService
- SessionService
- DiaryService
- ProgressService
- CrisisService
- AiService

Repository Layer

- UserRepository
- SessionRepository
- DiaryEntryRepository
- ChatMessageRepository

AI Integration

- RagContextBuilder
- SimpleVectorStore
- LlmClient
- CrisisDetector

Security

- JwtAuthenticationFilter
- JwtTokenProvider

Level 4: Code Diagrams (c4-code.puml)

- Class Diagram - AI Service Module showing AiService, RagContextBuilder, SimpleVectorStore, LlmClient, CrisisDetector and their relationships
- Sequence Diagram - Chat Message Flow showing interaction from User through CLI to AI response
- C) Sequence Diagram - Diary Entry with AI Analysis showing distortion analysis and reframing prompt generation

Diagram Requirements

- Use PlantUML format for all diagrams
- Follow C4 model notation standards (shapes, colors, relationships)
- Include a legend explaining notation
- Add meaningful labels to all relationships
- Export as PNG images AND include source .puml files

Grading Rubric (5 points)

Criterion	Points
Context Diagram (Level 1)	1.0
Container Diagram (Level 2)	1.0
Component Diagram (Level 3)	1.0
Code Diagrams (Level 4)	2.0

Submission Guidelines

Project Structure

```
1 digitaltherapyassistant/  
2 |— README.md  
3 |— pom.xml  
4 |— src/  
5 |   |— main/  
6 |       |— java/com/digitaltherapy/  
7 |           |— DigitalTherapyAssistantApplication.java  
8 |           |— cli/  
9 |               |— Command.java  
10 |               |— MenuHandler.java  
11 |               |— commands/  
12 |           |— controller/  
13 |           |— service/  
14 |               |— ai/  
15 |                   |— AiService.java  
16 |                   |— AiServiceImpl.java  
17 |                   |— RagContextBuilder.java  
18 |                   |— SimpleVectorStore.java  
19 |                   |— LlmClient.java  
20 |                   |— AnthropicClient.java  
21 |                   |— CrisisDetector.java  
22 |           |— repository/  
23 |           |— entity/  
24 |           |— dto/  
25 |           |— config/  
26 |           |— exception/  
27 |       |— resources/  
28 |           |— application.properties  
29 |           |— schema.sql  
30 |           |— data.sql
```

```

31 |         | knowledge-base/
32 |         | | distortions.json
33 |         | | cbt-techniques.json
34 |         | | crisis-protocols.json
35 |     | test/
36 |     | | java/com/digitaltherapy/
37 | docs/
38 | | architecture/
39 | | | c4-context.puml
40 | | | c4-context.png
41 | | | c4-container.puml
42 | | | c4-container.png
43 | | | c4-component.puml
44 | | | c4-component.png
45 | | | c4-code.puml
46 | | | c4-code.png
47 | | | README.md
48 | | api/
49 | | | openapi.json
50 | | erd/
51 | | | entity-relationship.png
52 | data/
53 | | vectors/

```

Appendix

Appendix A: Properties

application.properties

```

1 Application Configuration
2
3 spring.application.name=digital-therapy-assistant
4 server.port=8080
5
6 H2 Database Configuration
7
8 spring.datasource.url=jdbc:h2:file:./data/digitaltherapy_db
9 spring.datasource.driverClassName=org.h2.Driver
10 spring.datasource.username=sa
11 spring.datasource.password=
12
13 JPA/Hibernate Configuration
14
15 spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
16 spring.jpa.hibernate.ddl-auto=update
17 spring.jpa.show-sql=true
18 spring.jpa.properties.hibernate.format_sql=true
19
20 H2 Console (for development)
21
22 spring.h2.console.enabled=true
23 spring.h2.console.path=/h2-console
24
25 AI Service Configuration
26
27 ai.provider=anthropic
28 ai.api-key=${AI_API_KEY}
29 ai.model=claude-3-sonnet-20240229
30 ai.max-tokens=1024
31 ai.temperature=0.7
32
33 Vector Store Configuration
34
35 vectorstore.type=simpleVector

```

```

36 vectorstore.embedding-dimension=1536
37 vectorstore.similarity-metric=cosine
38 vectorstore.persistence-path=./data/vectors
39
40 JWT Configuration
41
42 jwt.secret=${JWT_SECRET:your-secret-key-here}
43 jwt.expiration=3600000
44 jwt.refresh-expiration=86400000
45
46 Logging Configuration
47
48 logging.level.com.digitaltherapy=DEBUG
49 logging.level.org.springframework=INFO
50 logging.level.org.hibernate.SQL=DEBUG

```

Appendix B: Knowledge Base Sample Files

distortions.json

```

1  {
2    "distortions": [
3      {
4        "id": "all-or-nothing",
5        "name": "All-or-Nothing Thinking",
6        "description": "Seeing things in black-and-white categories with
no middle ground.",
7        "examples": [
8          "If I'm not perfect, I'm a total failure.",
9          "Either I do it right or not at all."
10       ],
11       "reframingQuestions": [
12         "Is there a middle ground between these extremes?",
13         "Can you think of shades of gray in this situation?"
14       ]
15     },
16     {
17       "id": "catastrophizing",
18       "name": "Catastrophizing",
19       "description": "Expecting the worst possible outcome without
considering other possibilities.",
20       "examples": [
21         "If I make a mistake, I'll definitely get fired.",
22         "This headache must be a brain tumor."
23       ],
24       "reframingQuestions": [
25         "What's the most likely outcome?",
26         "How have similar situations turned out in the past?"
27       ]
28     },
29     {
30       "id": "mind-reading",
31       "name": "Mind Reading",
32       "description": "Assuming you know what others are thinking
without evidence.",
33       "examples": [
34         "They must think I'm incompetent.",
35         "She didn't smile because she hates me."
36       ],
37       "reframingQuestions": [
38         "What evidence do you have for this assumption?",
39         "Could there be other explanations for their behavior?"
40       ]
41     },
42     {
43       "id": "overgeneralization",

```

```
44     "name": "Overgeneralization",
45     "description": "Making broad conclusions based on a single
event.",
46     "examples": [
47         "I failed once, so I'll always fail.",
48         "Nobody ever listens to me."
49     ],
50     "reframingQuestions": [
51         "Is this truly always the case?",
52         "Can you think of exceptions to this pattern?"
53     ]
54 },
55 {
56     "id": "should-statements",
57     "name": "Should Statements",
58     "description": "Using rigid rules about how you or others should
behave.",
59     "examples": [
60         "I should always be productive.",
61         "They should know how I feel."
62     ],
63     "reframingQuestions": [
64         "Where does this rule come from?",
65         "What would happen if you replaced 'should' with 'prefer'?"
66     ]
67 }
68 ]
```